it as the associated data type). The file type imports the file name as a string, and copies the file to the output directory. I'd expect the use cases here being a need to import data directly into JavaScript so that the JavaScript can interact with it in a Rails app; however, I suspect that would be necessary only rarely.

–minify

The --minify option will minify the generated code, meaning the variable names and spacing will be changed to make the code smaller in size but harder to read. You would typically do this in production but not in development.

–splitting

The --splitting option is potentially useful if you have multiple entry points that share code. This option splits the common code into a shared file so that after the first entry point is loaded, future entry points only need to download the unique parts. This is potentially useful if you have, say, a lot of different pages with different entry points but common data and business logic shared. That said, it's probably something I would only try once I identified that I had a performance problem based on JavaScript loading.

–target

I've mentioned the --target attribute before—it takes an argument, which is a comma-separated list of targets. The default is --target=esnext, but you can specify the target in terms of either an ECMAScript version (--target=es2019) or a version of chrome, edge, firefox, node, or safari, as in --target=chrome97. You may specify more than one target, in which case esbuild will transform any language feature not supported by all the targets.

If your command-line options get complicated enough, you can offload them into a script that you can then run via node. Our existing command would be translated to this, which we could put in, say esconfig.js:

```
require('esbuild').build({
  entryPoints: ["app/javascript/*.*],
  bundle: true,
```